

PART 1 | IMPLEMENTATION

Patient Data Refresh Orchestrator

General design, operating model, and implementation approach

Design thesis Studies decide when a patient is due. EHR endpoints decide what that patient-level refresh costs. PostgreSQL makes queue state, deduplication, leases, and rate-limit admission one consistent system.

Executive summary

I designed the orchestrator around the patient because the supplied external contract is patient-level: `POST /patients/{id}/$updateData`. A patient may be enrolled in several studies, but those studies create freshness obligations rather than separate physical refresh calls. Coalescing the obligations into one job prevents paid duplicate work.

The implementation has four core operations: determine eligibility, schedule a job, claim work safely, and record the result. PostgreSQL is both the durable queue and the system of record so the correctness rules live next to the data they protect.

Goals and boundaries

In scope	Deliberate boundary
Real scheduling, queueing, worker coordination, retries, and rate limiting	The external EHR is mocked; clinical/FHIR resource modeling is not required
Patient/study freshness, patient/endpoint associations, durable job history	No separate Kafka/Redis queue and no second source of truth
Horizontal workers with crash recovery and duplicate prevention	Exercise-scale migrations and observability remain intentionally lightweight

Success criteria

- A due patient is scheduled once even when several studies are due or several schedulers run.
- Workers claim disjoint jobs in priority order and never send without rate-limit budget.
- Transient failures retry with backoff; permanent failures terminate; throttling does not spend the retry budget.
- A successful refresh advances every active study association in the same transaction as job completion.
- A worker can die without losing work, and a stale worker cannot overwrite a legitimate successor.

1. Architecture and responsibility boundaries

Component	Responsibility	Primary implementation
Scheduler	Find due patients, create jobs, spread load, fail expired work	src/scheduler.ts; domain/eligibility.ts; domain/schedule.ts
Worker pool	Claim, call EHR, poll, classify outcome, release lease	src/worker.ts; domain/claim.ts; domain/complete.ts
PostgreSQL	System of record, durable queue, dedupe, leases, token buckets	db/schema.sql; src/db.ts
Mock EHR	Simulate accepted, completed, transient, permanent, and 429 responses	src/mock-ehr/; src/ehr/client.ts
Admin API	Manual refresh and operational inspection	src/api.ts

Request flow

Stage	Action	Correctness property
1. Discover	Query active study enrollments whose cadence is due; group by patient	Several due studies coalesce into one refresh
2. Schedule	Insert one pending refresh_jobs row with priority, run_at, and deadline	Partial unique index makes duplicate prevention race-proof
3. Admit	Lock a due job with SKIP LOCKED; acquire every endpoint token atomically	No double-processing and no send without quota
4. Execute	POST once; store external request ID; release the worker lease	External latency does not pin workers
5. Poll	Reclaim at run_at and query status until terminal	Polling is durable and horizontally distributable
6. Commit	Complete job and advance active enrollment clocks in one transaction	Job truth and freshness truth cannot diverge

Mental model The due set triggers a refresh; the active set is satisfied by it. The job's coalesced_study_ids is an audit trail, not the list of records updated on success.

2. Data model

I kept the schema narrowly focused on orchestration. Patients and studies model the freshness obligation. Patients and EHR endpoints model the rate-limit cost. Refresh jobs carry the durable state machine.

Table	Role	Key design choice
patients	External patient identity	One durable orchestration subject
studies	Refresh cadence and base priority	Cadence belongs to the study
patient_studies	Enrollment and last_refresh_at	Active rows drive eligibility and are advanced on success
ehr_endpoints	Named source and per-minute limit	Endpoint is a quota/failure dimension
patient_endpoints	Patient-to-source association	One refresh consumes all associated budgets
refresh_jobs	Queue and job history	One active patient job; leases, retry budget, deadline, audit set
rate_limit_buckets	Durable token-bucket state	Rows are locked in a global endpoint order

Schema invariants

- refresh_jobs_one_active_per_patient is a partial unique index over pending and in_progress states.
- refresh_jobs_claim matches the live queue ordering: priority descending, then run_at ascending.
- Database now() is the sole clock for leases, deadlines, scheduling, and backoff, avoiding host-clock skew.
- Job states are constrained to pending, in_progress, completed, and failed; failure classes are separately constrained.
- Withdrawn study associations are never advanced by a successful refresh.

Why PostgreSQL

PostgreSQL supplies three load-bearing primitives in one place: FOR UPDATE SKIP LOCKED for concurrent claims, a partial unique index for deduplication, and transactions that couple the claim to multi-endpoint quota consumption. A separate queue would require cross-system consistency for the same invariants.

3. Concurrency and transaction design

Mechanism	Problem prevented	Boundary
FOR UPDATE SKIP LOCKED	Two workers taking the same due row	Claim transaction
Partial unique index + ON CONFLICT	Concurrent schedulers creating duplicate active jobs	Schedule insert
Lease with expiry	A crashed worker permanently owning work	One external HTTP call
locked_by fencing	A stale worker overwriting a successor	Every outcome write
Ascending endpoint_id locks	Deadlock during all-or-nothing quota admission	Rate-limit bucket acquisition

Atomic claim and quota admission

The worker first locks candidate jobs without waiting behind other workers. For each claimed patient, the same transaction loads the associated endpoint buckets, refills them using database time, and checks that every bucket has a token. Only fully admitted jobs remain leased; deferred jobs return to pending and extend their deadline for the time spent waiting on capacity.

Acquiring every endpoint token is all-or-nothing because the external request is patient-level. If a patient touches three endpoints and one has no budget, sending anyway would violate that endpoint's limit; spending two tokens and not sending would also be wrong. The transaction prevents both outcomes.

Asynchronous lifecycle

Observed outcome	State transition	Retry accounting
202 accepted	Store request ID, release lease, schedule poll	Counts one submitted attempt
Still retrieving	Release lease and schedule another poll	No attempt spent
Success	Complete job and advance all active enrollment clocks	Terminal
Transient failure	Return to pending with exponential backoff	Attempt spent; stop at budget
Permanent failure	Fail immediately	Terminal
429 / no local quota	Return to pending after cooldown	No attempt spent
Expired deadline	Fail as timeout	Bounds poison jobs and endless polling

Worker efficiency A worker holds a lease only while making one HTTP call. It does not occupy a worker slot while the EHR performs a long retrieval.

4. Scheduling, priority, and cost control

Eligibility

An active enrollment is due when `last_refresh_at` is absent or when `last_refresh_at` plus the study refresh interval is no later than database time. The query groups by patient, takes the maximum applicable priority, and records every due study ID for audit.

Priority and load spreading

Study priority is the baseline. Consent proximity raises urgency, while a user-triggered refresh applies an explicit high-priority override and bypasses jitter because a person is waiting. Scheduled work receives deterministic jitter within the configured window so large cohorts do not all become runnable at once.

Retry classification

Transient failures indicate a source that may recover, so they use capped exponential backoff with deterministic jitter. Permanent 4xx-style failures stop immediately. A 429 represents our traffic rate rather than job validity, so it delays the job without consuming a retry attempt. Poll observation failures retain the external request ID and try the poll again rather than paying for a second refresh.

Avoided calls

- Study coalescing makes one patient refresh satisfy several due obligations.
- Completion advances all active associations so a non-due study does not trigger a redundant call shortly afterward.
- The unique index prevents concurrent schedulers and manual triggers from duplicating active work.
- Stable idempotency keys protect ambiguous POST outcomes from becoming duplicate paid requests.
- Patient-level quarantine stops a permanently failing patient from being recreated every scheduler tick.

Known trade-offs

Trade-off	Why acceptable here	First scale fix
Hot token-bucket rows	Correct and simple at exercise worker counts	Lease quota to workers
Eligibility sweep computes due time	Small dataset and clear source of truth	Indexed <code>next_due_at</code>
Fixed polling interval	Easy to reason about and test	Adaptive polling or webhooks
No priority aging	Short demonstration queue	Age priority at high sustained load

5. Verification and handoff

I verify the design at three levels: database constraints, focused domain tests, and a live demonstration that runs the scheduler and multiple workers against PostgreSQL and the mock EHR.

Check	Evidence in the project
Build and type safety	TypeScript build and Vitest configuration
Eligibility and coalescing	test/eligibility.test.ts and test/schedule.test.ts
Atomic claiming and rate limits	test/claim.test.ts and test/robustness.test.ts
Retry state machine and fencing	test/complete.test.ts and test/worker.test.ts
End-to-end behavior	npm run demo reports outcomes and database invariants

Defects found by running the whole system

- A permanent job failure left the patient eligible, so the scheduler created a new paid job every tick. The fix quarantines recently failed patients while preserving a manual override.
- Initial load-spreading could place `run_at` after a deadline measured from creation. The fix measures the execution deadline from `run_at`.
- A throughput assertion conflicted with the configured endpoint limits. The test was replaced with assertions that match the actual capacity contract.

Authorship note

Transparent collaboration I established the problem framing, conceptual schema, component boundaries, requirements, operating rules, and written design direction. Claude was used primarily to implement and refine the PostgreSQL setup and the transactional SQL that coordinates the durable queue, claims, leases, rate limits, and completion paths. I reviewed, ran, tested, and corrected the resulting system. A separate contribution statement provides the detailed split.

This document is written in the first person because it records the design I directed and can explain. It does not claim that I manually typed every implementation line.